

Data Flow Diagram (DFD)

1 Introduction to Data Flow Diagrams

Data Flow Diagram (DFD) is a graphical representation that shows how data flows through a system, how it is processed, stored, and exchanged with external entities.

DFDs were developed in the early 1970s and are still widely used because they are:

- Simple and easy to learn
- Quick to sketch and understand
- Effective for system analysis and security modeling

A DFD abstracts a system into logical components instead of focusing on implementation details.

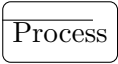
2 DFD Notations and Symbols

There are two primary notation systems used for constructing DFDs. While they represent the same concepts, their visual symbols differ slightly.

[Primary Notations]

- **Yourdon & DeMarco:** Uses circles for processes. Popular in system analysis.
- **Gane & Sarson:** Uses rounded rectangles (often with a header) for processes. Common in information systems.

2.1 Visual Comparison of Symbols

Component	Yourdon & DeMarco	Gane & Sarson
External Entity		
Entity		
Process		
Data Store		
Data Flow		
Trust		
Boundary		

3 Core Components of a DFD

A Data Flow Diagram consists of five fundamental elements.

3.1 External Entity

An **External Entity** is a producer or consumer of data that exists outside the system boundary.

Examples:

- People (user, administrator)
- Physical devices or sensors
- Other systems (computer-based systems)

Data must always originate from or be sent to an external entity.

3.2 Process

A **Process** represents a data transformation that converts input data into output data.

Examples of processes:

- Computing taxes, Determining area
- Formatting reports, Displaying graphs

- **Software Level:** DLLs, Components, Services, Web Services

Every process must have at least one input and one output.

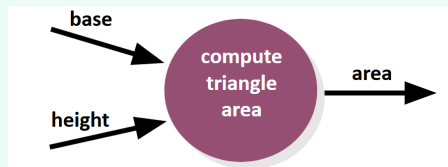
3.3 Data Flow

A **Data Flow** shows the movement of data between different components of the system.

Key idea: Data enters the system as input, flows through processes, and exits as output.

Examples:

- Function calls, API requests
- Network traffic, Remote procedure calls (RPC)



Example:

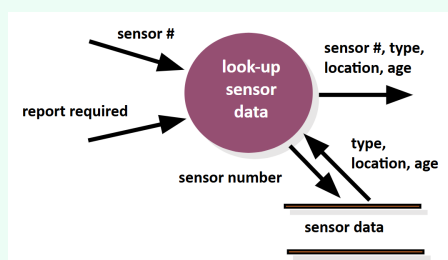
Base, Height → Compute Area → Area

3.4 Data Store

A **Data Store** is a repository where data is stored for later use.

Examples:

- Database, Registry
- Files, Shared memory
- Queues, Logs



Data cannot move directly between a data store and an external entity; it must pass through a process.

3.5 Trust Boundary

A **Trust Boundary** represents a point where the level of trust changes within a system.

Examples of Boundaries:

- Process boundary
- File system
- System boundary

It is commonly used in:

- Security modeling
- Threat analysis
- Cloud-based architectures

4 DFD Rules and Design Guidelines

- Every process must have at least one input and one output.
- Every data store must have at least one incoming and one outgoing data flow.
- Data must pass through a process before being stored.
- Data cannot flow directly between two external entities.
- All processes must connect to another process or a data store.

5 DFD Levels and Layering

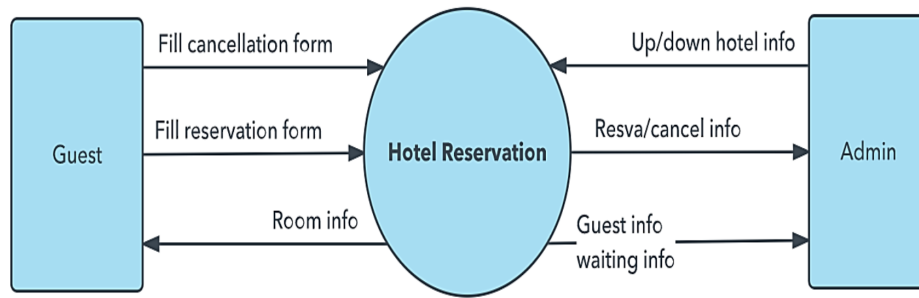
DFDs support progressive refinement by using multiple levels of detail.

5.1 Level 0: Context Diagram

DFD Level 0, also called the **Context Diagram**, provides a high-level overview of the entire system.

Characteristics:

- Represents the system as a single process
- Shows interactions with external entities
- Does not show internal data stores

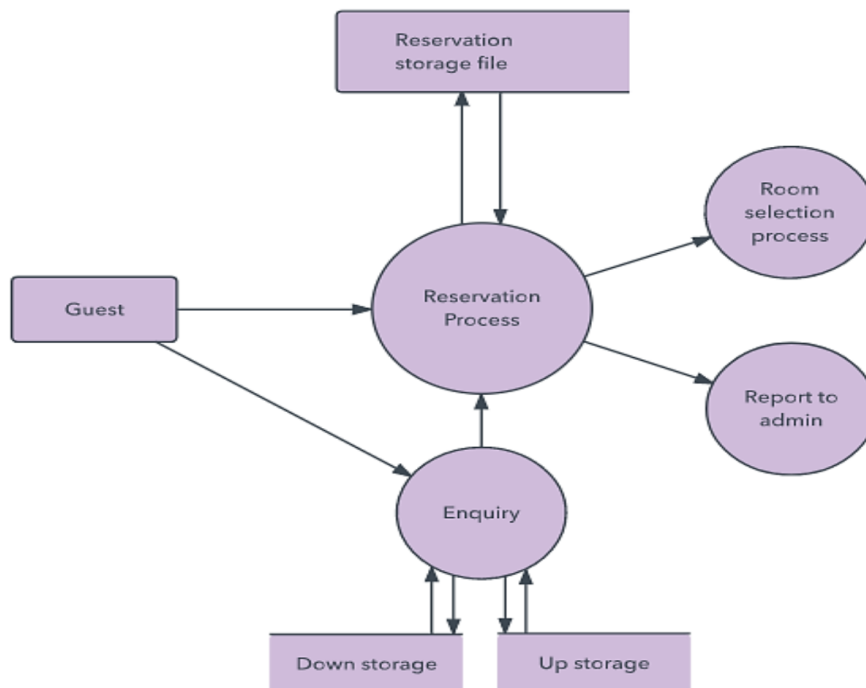


5.2 Level 1 DFD

DFD Level 1 expands the Context Diagram by breaking the main process into sub-processes.

Purpose:

- Identifies major system functions
- Shows data flows between internal processes
- Introduces data stores

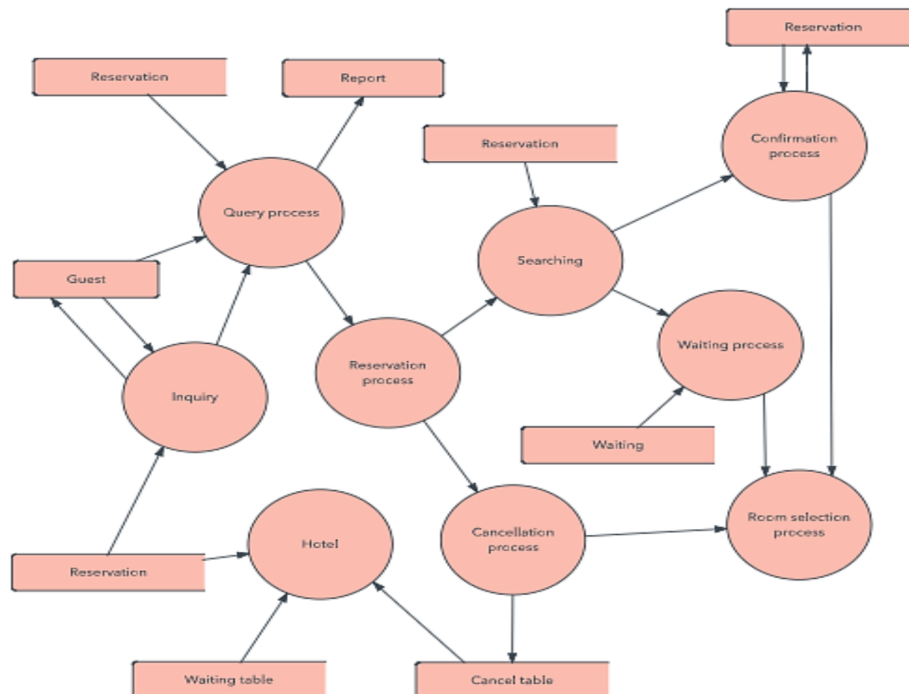


5.3 Level 2 DFD

DFD Level 2 further decomposes Level 1 processes into more detailed sub-processes.

Notes:

- Provides deeper system understanding
- May require additional textual explanation
- Used for complex systems



5.4 Beyond Level 2

Progression to Levels 3, 4, and beyond is possible, but:

Going beyond Level 3 is uncommon because excessive detail can reduce clarity and effectiveness.

6 Summary

- DFDs model how data moves through a system.
- They focus on **what** happens to data, not **how** it is implemented.
- Multiple levels allow gradual refinement from overview to detail.
- DFDs are essential in system analysis, design, and security modeling.